

Information Theory Project

A Simplified DEFLATE Compressor and Decompressor

1 Project Overview

In this project, you will implement a simplified compressor inspired by the main ideas inside DEFLATE.

The compressor works on files as byte streams. Even if the file is a text file, your program should treat it as bytes, not as Python strings.

The compression pipeline is:

Input Bytes $\rightarrow$ LZ77 Tokens $\rightarrow$ DEFLATE Symbols + Extra Bits $\rightarrow$ Canonical Huffman Bitstream

This is not a full .gz implementation. You are not required to implement the official GZIP header, CRC32, or official DEFLATE block structure.

The final correctness requirement is:

`decompress(compress(data)) = data`

byte-for-byte.

2 Stage 1: LZ77 Pattern Detection

2.1 Purpose

LZ77 finds repeated byte sequences.

Instead of writing the same bytes again, the compressor writes a reference to bytes that already appeared earlier.

For example:

1 `abcabcabcabc`

The first `abc` must be written normally. After that, the remaining `abcabcabc` can be described as:

Go back 3 bytes and copy 9 bytes.

So the LZ77 token stream is:

1 `Literal(97)`
2 `Literal(98)`
3 `Literal(99)`
4 `Match(length=9, distance=3)`

where 97, 98, and 99 are the byte values of `a`, `b`, and `c`.

2.2 Input and Output

The input to Stage 1 is the original byte array:

$$b_0, b_1, b_2, \dots, b_{n-1}$$

where each byte is between 0 and 255.

The output is a list of LZ77 tokens.

There are only two token types:

`Literal(byte)`

and:

```
Match(length, distance)
```

A literal means:

Output this byte directly.

A match means:

Go back `distance` bytes in the already reconstructed output and copy `length` bytes.

2.3 Required Constants

Use the following constants:

WINDOW_SIZE = 32768

$$\text{MIN_MATCH} = 3$$

MAX_MATCH = 258

Therefore, every match must satisfy:

$$3 \leq \text{length} \leq 258$$

and:

$$1 \leq \text{distance} \leq 32768$$

If the best repeated sequence has length 1 or 2, emit literals instead of a match.

2.4 Meaning of Distance

Suppose the decoder has already reconstructed:

1	abc
---	-----

Now it receives:

```
Match(length=9, distance=3)
```

The decoder goes back 3 bytes from the current output position. That points to the start of `abc`. Then it copies 9 bytes.

The result becomes:

1	abcabcabcabc
---	--------------

2.5 Overlapping Matches

LZ77 allows overlapping matches.

For example:

1	aaaaaaaaaa
---	------------

can be represented as:

```
1 Literal(97)
```

```
2 Match(length=9, distance=1)
```

The decoder first outputs one **a**. Then the match says: go back 1 byte and copy 9 bytes. As the decoder copies new **a** bytes, those newly copied bytes also become available for the rest of the same match.

So the decompressor must copy matches byte by byte.

2.6 Match Selection Rule

At every input position, your compressor must choose the token using this rule:

1. Search only inside the previous 32768 bytes.
2. Find the longest match starting at the current position.
3. If multiple matches have the same length, choose the smaller distance.
4. If the best match length is at least 3, emit a match.
5. Otherwise, emit a literal.

2.7 Simple Hash-Based Match Search

A naive implementation checks every previous position in the last 32768 bytes. This is easy to understand but can be slow.

To make matching faster, use this simple idea:

A valid match must start with at least 3 equal bytes.

So instead of checking all previous positions, group positions by their next 3 bytes.

For example, in:

```
1 abcxxxabcabc
```

the 3-byte sequence `abc` appears at positions 0, 6, and 9.

So when the current position starts with `abc`, it is useful to check only previous positions that also started with `abc`.

2.8 The Dictionary of 3-Byte Keys

Use a dictionary:

`table`

where:

`table[key] = list of previous positions where this key appeared`

The key is simply the next 3 bytes.

For example, if:

```
1 data[i:i+3] == b"abc"
```

then the key may be:

```
1 (97, 98, 99)
```

or the bytes object:

```
1 b"abc"
```

2.9 How the Search Works

At position i :

1. If fewer than 3 bytes remain, emit literals.
2. Otherwise, compute the key `data[i:i+3]`.
3. Look up previous positions stored in `table[key]`.
4. Check candidates from newest to oldest.
5. Ignore candidates where:
$$i - \text{candidate} > 32768$$
6. For each remaining candidate, count how many bytes match, up to 258.
7. Choose the longest match.

To keep the search efficient, check at most:

$$\text{MAX_CANDIDATES} = 64$$

previous positions for each current position.

2.10 Updating the Table

Whenever the compressor passes a position, insert its 3-byte key into the table.

If the compressor emits a literal, insert the current position.

If the compressor emits a match of length L , insert all positions covered by that match:

$$i, i + 1, i + 2, \dots, i + L - 1$$

as long as each position has at least 3 bytes remaining.

This helps future matches.

2.11 Stage 1 Requirements

Your Stage 1 implementation must:

1. Treat the input as bytes.
2. Produce only `Literal(byte)` and `Match(length,distance)` tokens.
3. Use the constants `WINDOW_SIZE = 32768`, `MIN_MATCH = 3`, and `MAX_MATCH = 258`.
4. Never emit a match with length less than 3.
5. Never emit a match with length greater than 258.
6. Never emit a match with distance greater than 32768.
7. Support overlapping matches during decompression.
8. Use a dictionary of 3-byte keys, or an equivalent efficient structure, to avoid checking every previous position.

3 Stage 2: DEFLATE Symbols and Extra Bits

3.1 Purpose

Stage 1 produces LZ77 tokens.

Stage 2 converts these tokens into symbols suitable for Huffman coding.

A literal is already a symbol.

A match must be converted into:

length symbol + length extra bits

and:

distance symbol + distance extra bits

3.2 Input and Output

The input to Stage 2 is the LZ77 token stream.

The output is an event stream.

There are three event types:

`LiteralEvent(symbol)`

`MatchEvent(length_symbol, length_extra, distance_symbol, distance_extra)`

`EndEvent(256)`

Here, `length_extra` and `distance_extra` are bit strings.

For example, the length extra bits may be:

01

and the distance extra bits may be:

1

In implementation, these can be represented as strings such as "01" and "1", or as a pair (*value*, *bit_count*). The concept is the same: each extra field is one bit string.

3.3 Literal/Length Symbols

The literal/length alphabet has 286 symbols:

Symbol Range	Meaning
0–255	Literal byte values
256	End-of-block marker
257–285	Length symbols for matches

A literal byte *b* becomes:

`LiteralEvent(b)`

Examples:

Literal(65) → *LiteralEvent*(65)

$$Literal(97) \rightarrow LiteralEvent(97)$$

At the end of the event stream, append:

$$EndEvent(256)$$

This end marker tells the decoder where the compressed payload stops.

3.4 Distance Symbols

The distance alphabet has 30 symbols:

$$0, 1, 2, \dots, 29$$

These represent match distances from 1 to 32768.

3.5 Meaning of Extra Bits

Some length and distance symbols represent one exact value.

For example:

$$9 \rightarrow \text{length symbol } 263$$

No extra bits are needed.

Other symbols represent a range.

For example:

$$19, 20, 21, 22 \rightarrow \text{length symbol } 269$$

So symbol 269 only tells the decoder that the length is in the range 19–22.

The extra bits identify the exact value:

$$00 \rightarrow 19$$

$$01 \rightarrow 20$$

$$10 \rightarrow 21$$

$$11 \rightarrow 22$$

Therefore:

$$20 \rightarrow \text{symbol } 269 + \text{extra bits } 01$$

3.6 Length Table

Use the following table exactly.

Length Range	Length Symbol	Extra Bits
3	257	0
4	258	0
5	259	0
6	260	0
7	261	0
8	262	0

9	263	0
10	264	0
11–12	265	1
13–14	266	1
15–16	267	1
17–18	268	1
19–22	269	2
23–26	270	2
27–30	271	2
31–34	272	2
35–42	273	3
43–50	274	3
51–58	275	3
59–66	276	3
67–82	277	4
83–98	278	4
99–114	279	4
115–130	280	4
131–162	281	5
163–194	282	5
195–226	283	5
227–257	284	5
258	285	0

3.7 Length Encoding Arrays

Use these arrays for implementation:

```

1 length_base = [
2     3, 4, 5, 6, 7, 8, 9, 10,
3     11, 13, 15, 17,
4     19, 23, 27, 31,
5     35, 43, 51, 59,
6     67, 83, 99, 115,
7     131, 163, 195, 227,
8     258
9 ]
10
11 length_extra = [
12     0, 0, 0, 0, 0, 0, 0, 0, 0,
13     1, 1, 1, 1,
14     2, 2, 2, 2,
15     3, 3, 3, 3,
16     4, 4, 4, 4,
17     5, 5, 5, 5,
18     0
19 ]

```

For index i , the length symbol is:

$$257 + i$$

To encode a length, find the row where:

$$base \leq length \leq base + 2^{extra_count} - 1$$

Then compute:

$$extra_value = length - base$$

and write that value using exactly `extra_count` bits.

3.8 Distance Table

Use the following table exactly.

Distance Range	Distance Symbol	Extra Bits
1	0	0
2	1	0
3	2	0
4	3	0
5–6	4	1
7–8	5	1
9–12	6	2
13–16	7	2
17–24	8	3
25–32	9	3
33–48	10	4
49–64	11	4
65–96	12	5
97–128	13	5
129–192	14	6
193–256	15	6
257–384	16	7
385–512	17	7
513–768	18	8
769–1024	19	8
1025–1536	20	9
1537–2048	21	9
2049–3072	22	10
3073–4096	23	10
4097–6144	24	11
6145–8192	25	11
8193–12288	26	12
12289–16384	27	12
16385–24576	28	13
24577–32768	29	13

3.9 Distance Encoding Arrays

```

1 distance_base = [
2     1, 2, 3, 4,
3     5, 7,
4     9, 13,
5     17, 25,
6     33, 49,
7     65, 97,
8     129, 193,
9     257, 385,
10    513, 769,
11    1025, 1537,
12    2049, 3073,

```

```

13     4097, 6145,
14     8193, 12289,
15     16385, 24577
16 ]
17
18 distance_extra = [
19     0, 0, 0, 0,
20     1, 1,
21     2, 2,
22     3, 3,
23     4, 4,
24     5, 5,
25     6, 6,
26     7, 7,
27     8, 8,
28     9, 9,
29     10, 10,
30     11, 11,
31     12, 12,
32     13, 13
33 ]

```

For distances, the distance symbol is the array index.

To encode a distance, find the row where:

$$base \leq distance \leq base + 2^{extra_count} - 1$$

Then compute:

$$extra_value = distance - base$$

and write it using exactly `extra_count` bits.

3.10 Stage 2 Example

Stage 1 output:

```

1 Literal(97)
2 Literal(98)
3 Literal(99)
4 Match(length=20, distance=6)

```

The literals become:

$$LiteralEvent(97), \quad LiteralEvent(98), \quad LiteralEvent(99)$$

Length 20 belongs to the length range 19–22:

$$20 \rightarrow \text{length symbol } 269 + \text{extra bits } 01$$

Distance 6 belongs to the distance range 5–6:

$$6 \rightarrow \text{distance symbol } 4 + \text{extra bits } 1$$

So the match becomes:

$$MatchEvent(269, 01, 4, 1)$$

Finally append:

$$EndEvent(256)$$

So the Stage 2 event stream is:

```

1 LiteralEvent(97)
2 LiteralEvent(98)
3 LiteralEvent(99)
4 MatchEvent(269, "01", 4, "1")
5 EndEvent(256)

```

4 Stage 3: Canonical Huffman Coding

4.1 Purpose

Stage 3 compresses the symbols produced by Stage 2.

There are two separate Huffman alphabets:

1. Literal/length symbols: 0–285.
2. Distance symbols: 0–29.

Only symbols are Huffman encoded.

Extra bits are not Huffman encoded.

For a match, the payload order is:

Huffman(length_symbol)

then:

length_extra

then:

Huffman(distance_symbol)

then:

distance_extra

4.2 Frequency Counting

Create two frequency arrays:

lit_freq[286]

dist_freq[30]

For each event:

- `LiteralEvent(s)` increases `lit_freq[s]`.
- `MatchEvent(len_sym, len_extra, dist_sym, dist_extra)` increases `lit_freq[len_sym]` and `dist_freq[dist_sym]`.
- `EndEvent(256)` increases `lit_freq[256]`.

Extra bits are not counted because they are raw bits, not Huffman symbols.

4.3 Building Huffman Lengths

For each alphabet, build a normal Huffman tree from frequencies.

The only value you need from the tree is the code length of each symbol.

If a symbol is unused, its length is 0.

If an alphabet contains exactly one used symbol, assign it length 1.

To keep results deterministic, when two nodes have equal frequency, break ties using the smallest symbol value inside the subtree.

4.4 Canonical Huffman Rule

After computing code lengths, convert them to canonical Huffman codes.

Canonical Huffman coding means:

The actual bit codes are determined only by the code lengths and the symbol values.

The rule is:

1. Count how many symbols have each length.
2. Compute the first code for each length.
3. Assign codes to symbols in increasing symbol order.

Use the following code-generation logic:

```
1 count = [0] * 16
2 for length in lengths:
3     count[length] += 1
4
5 count[0] = 0
6
7 next_code = [0] * 16
8 code = 0
9
10 for bits in range(1, 16):
11     code = (code + count[bits - 1]) << 1
12     next_code[bits] = code
13
14 for symbol in range(len(lengths)):
15     length = lengths[symbol]
16     if length != 0:
17         symbol_code[symbol] = next_code[length]
18         next_code[length] += 1
```

4.5 Canonical Example

Suppose the literal/length code lengths are:

Symbol	Length
97	3
98	3
99	2
256	2
263	2

The symbols are ordered by length, then by value:

99, 256, 263, 97, 98

The canonical codes are:

99 → 00

256 → 01

263 → 10

97 → 110

98 → 111

The decompressor can reconstruct these exact codes using only the code lengths.

5 Stage 4: Custom Header and Payload

5.1 Purpose

The compressed payload uses Huffman codes.

Therefore, the decompressor must know the Huffman code lengths before it can decode the payload. Your file format stores:

1. the literal/length code-length table,
2. the distance code-length table,
3. the Huffman-coded payload.

5.2 Compressed File Layout

The compressed file must contain exactly these fields in this order:

Field	Size	Description
LIT_BW	4 bits	Bit-width used for literal/length code lengths
DIST_BW	4 bits	Bit-width used for distance code lengths
LIT_TABLE	$286 \times \text{LIT_BW}$ bits	Code lengths for symbols 0–285
DIST_TABLE	$30 \times \text{DIST_BW}$ bits	Code lengths for symbols 0–29
PAYLOAD	variable	Huffman-coded data plus raw extra bits

5.3 Computing Table Bit-Widths

Let M be the largest code length in a table.

The bit-width is:

$$BW = \begin{cases} 0, & M = 0 \\ \lfloor \log_2(M) \rfloor + 1, & M > 0 \end{cases}$$

Examples:

Maximum length	Bit-width
0	0
1	1
2 or 3	2
4–7	3
8–15	4

LIT_BW is computed from the maximum literal/length code length.

DIST_BW is computed from the maximum distance code length.

If DIST_BW = 0, the distance table has no bits because no distance symbols are used.

5.4 Bit Order

All bits in this assignment are written most-significant bit first.

For example:

$$6 = 110_2$$

Writing 3 bits gives:

$$1, 1, 0$$

A more important example is:

$$2 = 10_2$$

Writing 4 bits gives:

$$0, 0, 1, 0$$

because the value must be padded to exactly 4 bits:

$$0010_2$$

5.5 Payload Writing Rule

After the header, write the event stream.

For:

$$LiteralEvent(s)$$

write:

$$Huffman(s)$$

For:

$$MatchEvent(len_sym, len_extra, dist_sym, dist_extra)$$

write:

$$Huffman(len_sym)$$

then:

$$len_extra$$

then:

Huffman(dist_sym)

then:

dist_extra

For:

EndEvent(256)

write:

Huffman(256)

Finally, pad the last byte with zeros if the number of bits is not a multiple of 8.

6 Decompression

6.1 Header Reading

The decompressor first reads:

LIT_BW

and:

DIST_BW

Then it reads 286 literal/length code lengths, each using *LIT_BW* bits.

Then it reads 30 distance code lengths, each using *DIST_BW* bits.

From these lengths, it reconstructs the canonical Huffman codes.

6.2 Payload Decoding

The decoder repeatedly reads one literal/length Huffman symbol.

There are three cases:

- If the symbol is 0–255, append it to the output as one byte.
- If the symbol is 256, stop.
- If the symbol is 257–285, decode a match.

To decode a match:

1. Use the length table to get the base length and number of length extra bits.
2. Read the length extra bits and compute the actual length.
3. Decode one distance Huffman symbol.
4. Use the distance table to get the base distance and number of distance extra bits.
5. Read the distance extra bits and compute the actual distance.
6. Perform the LZ77 copy into the output buffer.

7 Complete Worked Example

7.1 Input

```
1  abcabcbcabcb
```

Byte values:

97, 98, 99, 97, 98, 99, 97, 98, 99, 97, 98, 99

7.2 Stage 1 Output

```
1  Literal(97)
2  Literal(98)
3  Literal(99)
4  Match(length=9, distance=3)
```

7.3 Stage 2 Output

The literals become:

LiteralEvent(97), LiteralEvent(98), LiteralEvent(99)

Length 9 becomes:

length_symbol = 263

with no extra bits.

Distance 3 becomes:

distance_symbol = 2

with no extra bits.

Append:

EndEvent(256)

So the event stream is:

```
1  LiteralEvent(97)
2  LiteralEvent(98)
3  LiteralEvent(99)
4  MatchEvent(263, "", 2, "")
5  EndEvent(256)
```

7.4 Stage 3 Frequencies

Literal/length frequencies:

Symbol	Frequency
97	1
98	1
99	1
263	1
256	1

Distance frequencies:

Symbol	Frequency
2	1

One valid deterministic Huffman construction gives:

Symbol	Code Length
97	3
98	3
99	2
256	2
263	2

Distance symbol 2 receives length 1.

7.5 Canonical Codes

Literal/length codes:

$$99 \rightarrow 00$$

$$256 \rightarrow 01$$

$$263 \rightarrow 10$$

$$97 \rightarrow 110$$

$$98 \rightarrow 111$$

Distance code:

$$2 \rightarrow 0$$

7.6 Payload Bits

The payload is:

$$\textit{LiteralEvent}(97) \rightarrow 110$$

$$\textit{LiteralEvent}(98) \rightarrow 111$$

$$\textit{LiteralEvent}(99) \rightarrow 00$$

$$\textit{MatchEvent}(263, \text{""}, 2, \text{""}) \rightarrow 10\ 0$$

$$\textit{EndEvent}(256) \rightarrow 01$$

Therefore:

$$\text{payload bits} = 1101110010001$$

The final byte is padded with zeros if needed.

8 Submission Format

Submit a single `.zip` file containing all source code files needed to run your compressor and decompressor.

The submitted folder must include a runnable file named:

```
1 main.py
```

Your program must support the following terminal arguments.

To compress a file:

```
1 python main.py -c filename
```

This should create the compressed file in the same directory as the input file. The compressed output file must be named by appending `.sdf1` to the original filename. For example, compressing `data.txt` must produce `data.txt.sdf1`. When decompressing `data.txt.sdf1`, the output should be named `data.txt`.

To decompress a file:

```
1 python main.py -d filename
```

This should create the decompressed file in the same directory as the compressed input file.